Q-Day Prize Submission Brief

Author: Giancarlo Lelli | **Contact**: gcarlo.lelli@gmail.com

Overview

This submission implements Shor's algorithm for the Elliptic Curve Discrete Logarithm Problem (ECDLP) on IBM Quantum hardware, targeting the Q-Day Prize challenge curves ($y^2 = x^3 + 7$ over F_p). The solver successfully recovered private keys for challenge curves up to 10-bit on real quantum hardware using only the IBM Quantum open-instance free tier (10 minutes/month).

Algorithm

The solver implements the two-register variant of Shor's algorithm for ECDLP. Given a generator G and public key Q = dG on an elliptic curve of order n, the circuit prepares counting registers |j>, |k> in superposition, computes |j>|k>|jG + kQ> via controlled point additions, applies inverse QFT, and extracts d from the relation j + kd = r (mod n). No knowledge of d is used in circuit construction.

Oracle Strategies

Five oracle strategies are implemented, selected automatically or via `--oracle`:

Dense Unitary (n_bits <= 6): Each controlled "add S" is a $2^{n+1} \times 2^{n+1}$ permutation matrix. Used for 4-bit and 6-bit challenges.

Efficient Permutation (n_bits > 6): Each "add S" permutation is cycle-decomposed into transpositions using CNOT-reduction + V-chain MCX. O(N * n) gates per addition. Used for 8-bit through 10-bit challenges.

Coordinate Oracle: Point register holds actual (x, y) coordinates instead of group indices. Verified at 4-bit and 6-bit.

Arithmetic Oracle: QFT-based modular arithmetic primitives (Beauregard adders, modular multiply/inverse) with $O(n^3)$ asymptotic scaling -- the foundation for polynomial-scaling point addition at 256-bit on future hardware.

Semiclassical Phase Estimation (`--oracle google`): Inspired by Babbush et al. (2026) and Griffiths & Niu (1996). Replaces the two multi-qubit counting registers + bulk inverse QFT with two single recycled qubits and classically-conditioned phase corrections via dynamic circuits (`reset`, `if_test`). Achieves 46-59% qubit reduction. Successfully verified on IBM Heron r2 hardware at 4-bit (5 qubits), 6-bit (7 qubits), and 7-bit (14 qubits). At 8-bit+, the classical feedback synchronization overhead on IBM QPUs exceeds the time budget; the standard permutation approach (which runs as a single continuous batch) remains practical at that scale.

Results

All executions ran on IBM Quantum hardware via `qiskit-ibm-runtime` SamplerV2, transpiled at optimization level 3.

Challenge	Strategy	Qubits	2Q Gates	Shots	Backend	Recovered d
4-bit	Dense unitary	11	774	8,192	ibm_torino	**6**
4-bit	Coordinate oracle	24	6,449	8,192	ibm_kingston	**6**
4-bit	Arithmetic oracle	24	6,477	8,192	ibm_torino	**6**
4-bit	Semiclassical PE	5	747	256	ibm_kingston	**6**
6-bit	Dense unitary	17	23,471	8,192	ibm_torino	**18**
6-bit	Coordinate oracle	36	95,254	8,192	ibm_kingston	**18**
6-bit	Semiclassical PE	7	23,256	256	ibm_kingston	**18**
7-bit	Semiclassical PE	14	127,918	256	ibm_kingston	**56**
8-bit	Efficient perm.	32	294,628	8,192	ibm_kingston	**103**
9-bit	Efficient perm.	36	887,544	8,192	ibm_torino	**135**
10-bit	Efficient perm.	40	2,049,138	1,024	ibm_torino	**165**

Every recovered key was verified classically: d * G = Q. All source code, challenge curves, and execution logs are included in the repository.

Key Findings

Noise robustness: At 8-bit+, circuit fidelity is astronomically small (~10^{-644} at 8-bit) yet the algorithm recovers correct keys. Shor's post-processing is robust to noise: correct signal shots always vote for the true d, while noise scatters uniformly across n candidates. Even 3-4 signal shots out of thousands suffice.

Dynamic circuits on NISQ hardware: The semiclassical PE strategy demonstrates qubit-recycled phase estimation -- a technique central to Google's 2026 secp256k1 resource estimates -- working on real IBM hardware. The 7-bit run (14 qubits, 128K CZ gates, 14 classical feedback points) is, to our knowledge, among the first demonstrations of semiclassical Shor ECDLP on superconducting hardware.

Reproducibility

```
git clone https://github.com/GiancarloLelli/quantum.git
cd quantum && python -m venv . && Scripts/Activate.ps1
pip install -r requirements.txt
python projecteleven.py --challenge 4 --token YOUR_IBM_TOKEN --backend ibm_torino
```

Requires only `qiskit` and `qiskit-ibm-runtime`. No local simulator, no classical pre-computation of the secret key.